



Modelos e Agentes de IA dentro do cenário de Sistema Elétrico de Potência

Visão Geral: Ecossistema ONS-PLC-PV

Este documento reuniu todo ecossistema criado durante o estágio que integra automação para o setor elétrico, ferramentas de produtividade e gestão de conhecimento pessoal.



■ 1. Automações e Projetos ONS/PLC

Focada em ferramentas técnicas para o Operador Nacional do Sistema (ONS), especialmente ETL e RPA.

- **ONS-PLC-PV-CONTROLE-E-AUTOMACAO**
- **AUTOMACÕES ONS - PLC:** Scripts de controle de hardware e lógica industrial.
- **Análise De Dados - Python + Excel + SQL:** Pipelines de BI e relatórios automáticos.
- **Relatórios QUARTO e PDF:** Geração de documentos técnicos dinâmicos.
- **Meu-Segundo-Cerebro-2026:** Base de conhecimento integrada.
- **Análise:** Demonstra domínio de integração entre software e infraestrutura crítica (IoT/Indústria).
- **Palkia-PDF-extractor**
- **src:** Lógica core de extração de dados MUST.
- **tools:** Utilitários de suporte ao processamento de PDFs.
- **Plugin Organon Notepad++:** Extensão para facilitar a visualização de dados extraídos.
- **Análise:** Projeto de ETL de alta precisão. Ideal para aplicar **TDD** na validação de parsers complexos.
- **Repopulation-With-Elite-Set (UFF)**
- **app.py & src:** Implementação de Algoritmos Genéticos para Redes Elétricas.
- **docs-sphinx:** Documentação técnica rigorosa.
- **Análise:** O projeto de maior complexidade matemática. Perfeito para integração com **Quarto.md** e **Sympy** para documentar as equações diferenciais dos sistemas elétricos.
- **ETL de LTs/Trafos:** Processamento de dados de Linhas de Transmissão e Transformadores.
- **RPA SAPECP:** Automação de processos para o sistema SAPECP.s
- **Montagem de Casos Mensais:** Scripts para preparação de decks e casos do AnaREDE.
- **Salamance-Qt6-ETL:** Interface gráfica moderna para extração de dados.

■ ■ 2. Ferramentas de Sistema e CLI (C3PO)

Utilitários de linha de comando e assistência que compõem um "sistema operacional" personalizado.

- **Copilot_C3PO:** O "cérebro" do sistema com agentes e backend Flask.
- **CLI Multi-stack:** Implementações em Rust, C++ e Python.
- **Integração TTS:** Notificações por voz e popups automáticos para tarefas.

■ ■ 3. Arquitetura Backend (Pikachu(Flask) -> Raichu(FastAPI))

Ecossistema integrado que conecta Flask (servidor), CLI (cliente) e TTS.

- **Servidor Flask:** Centraliza notificações e processamento.
- **Notificações:** Gerenciador centralizado de TTS + Windows Toast.
- **Interface:** CLI interativa v4 para gestão de tarefas e arquivos.

■ Ecossistema Python, Engenharia e Automação

- **Pikachu-Flask-Server**
- **pikachu-API:** Núcleo de serviços REST unificados.
- **batcaverna:** Ambiente de ferramentas internas e scripts de utilidade.
- **automate:** Scripts de automação de workflow.
- **Análise:** Estrutura de "Plataforma Única". Ótimo para aplicar **SOLID** na separação de serviços de terceiros (NASA, Pokémon).

■ 4. Gestão de Conhecimento e Metodologias Ágeis

Área dedicada à organização pessoal e evolução contínua (Segundo Cérebro).

- **Obsidian**: Repositório integrado como cofre de notas Markdown.
- **Kanban 2026**: Roadmaps de aprendizado Fullstack e metas técnicas.
- **PV Core**: Notas sobre Mente, Corpo e Rotina.
- **Jedi-CyberPunk**
- Configurações temáticas (Batman, Anakin, Gohan Mode) e uso do **Obsidian** para gestão de conhecimento.

SCRUM + Kanban + XP (Extreme Programming)

- **Kanban para Fluxo:** Visualize o trabalho. Use colunas como `Backlog`, `Development`, `Testing`, `Deployment`.
- **XP para Qualidade:**
- **Pair Programming:** Mesmo sozinho, use o Gemini CLI para "codar em par".
- **Small Releases:** No projeto `Pikachu`, faça deploys pequenos e frequentes.
- **Refactoring:** No projeto `Palkia`, refatore o código assim que o teste passar.

SOLID e TDD

- **S (Single Responsibility):** Cada classe no `Pikachu-API` deve ter apenas um motivo para mudar.
- **O (Open/Closed):** Extenda o `Fleting Framework` sem modificar o código core.
- **TDD (Test-Driven Development):** Comece escrevendo o teste que falha. No `Network-Tunnel`, teste a criação de um túnel antes de implementar a lógica.

Design Patterns e MVVM

- **Factory Pattern:** Use para criar diferentes tipos de exportadores no `Palkia`.
- **Strategy Pattern:** No projeto `ONS`, use para trocar algoritmos de análise de dados dinamicamente.
- **MVVM (Model-View-ViewModel):** Essencial no `getx-for-qt6`. A `View` (UI) não deve ter lógica; ela apenas observa o `ViewModel`, que por sua vez manipula o `Model`.

Programação Frontend: 2026

■ Ecossistema JavaScript / TypeScript / Fullstack

- **Network-Tunnel-Go-VM**
- **server/client/shared:** Arquitetura fullstack completa para gestão de túneis Cloudflare.
- *Análise:* Uso de **Drizzle ORM** e **Vite** indica domínio de stacks de alta performance.
- **Gohan-treinamentos-web-app**
- **gohan-treinamentos-app:** Frontend em React/MUI.
- **backend:** Lógica de suporte e integração com IA.
- *Análise:* Foco em **UX/UI** e interação com modelos de linguagem (IA Personal Agent).
- **Websites-Astro-Templates**
- Templates variados usando Astro para unir React, Svelte e Vue.
- *Análise:* Demonstra versatilidade em múltiplos frameworks frontend (Island Architecture).
- **dashboard-website-template**
- **futuristic-ons-dashboard:** UI temática para o sistema ONS.
- **streamlitprojecttemplate:** Dashboards rápidos para ciência de dados.
- **dashboard-must-tauri-app:** Versão desktop com Tauri.

■ Mobile e Desktop Nativo

- **my-flutter-getx-template-app**
- Templates para apps de produtividade e dashboards usando **GetX** e **SQLite**.
- *Análise:* Segue o padrão **MVVM** rigorosamente no Flutter.
- **Gohan-Treinamentos-Desktop-App**
- Migração para Tauri V2 com Rust.
- *Análise:* Busca por performance nativa e segurança no ambiente desktop.
- **getx-for-qt6 / Fleting Framework**
- **fleting:** Micro-framework baseado em Flet.
- **pvrv_advanced:** Módulos de cálculo e gestão avançada.
- **eti-palkia-pdf-extractor:** Versão integrada do extrator.
- **meuprojetonovo_qt6:** Template para aplicações desktop modernas.
- *Análise:* Foco em criar ferramentas de produtividade para outros desenvolvedores (Mentalidade de Arquiteto).

■ 5. Laboratório e Data Science

Exploração de novas tecnologias e análise de desempenho.

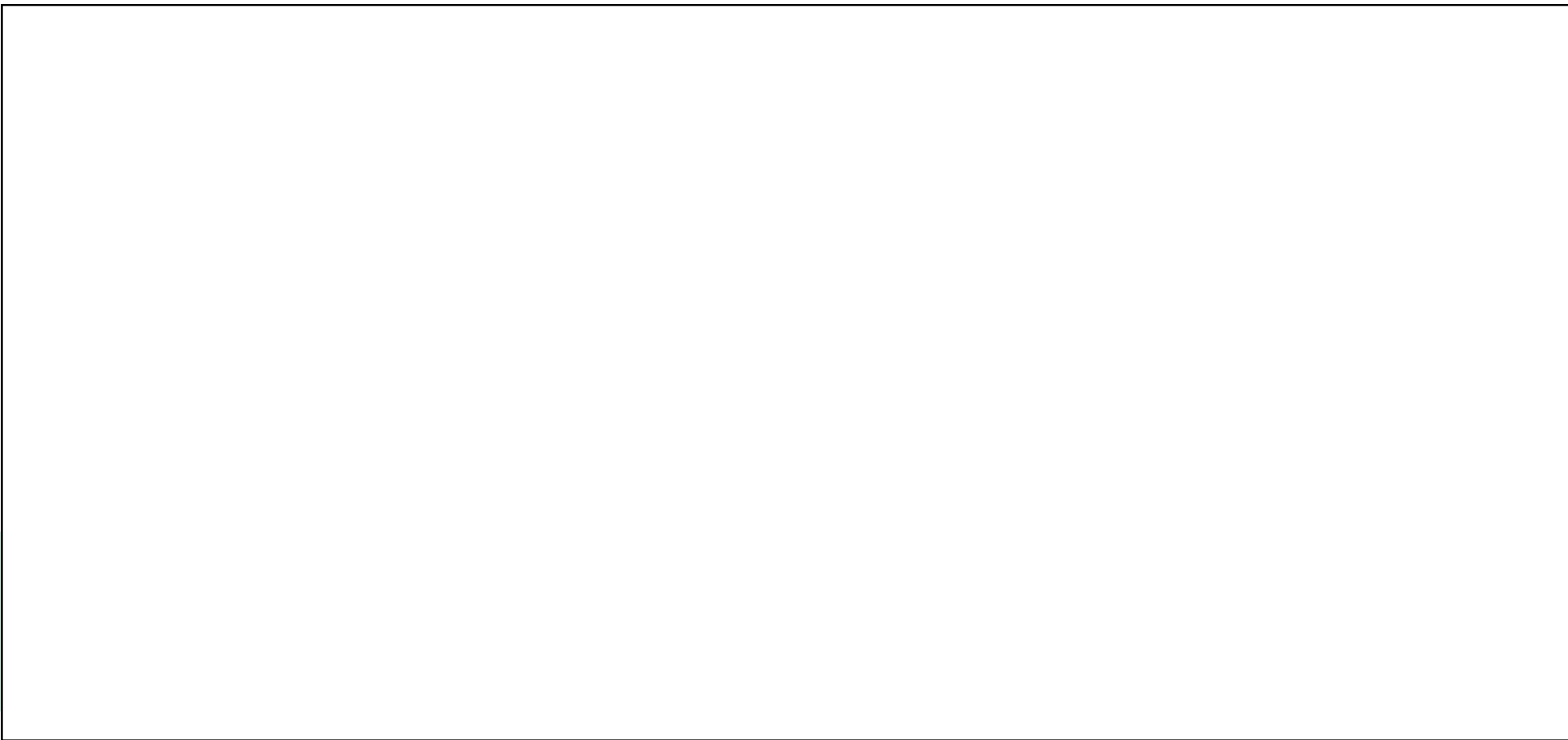
- **Estudos:** Comparativos Polars vs Pandas, Integração Rust/Python.
- **Notebooks:** Relatórios gerados via Quarto e Jupyter para análise de commits.

■ 6. Novo: RAG Chatbot com Gemini

TODO

Assistente inteligente para análise de documentos técnicos.

- **PySide6 UI:** Interface amigável para upload e chat.
- **RAG Engine:** Analisa PDFs e Excel em tempo real.
- **Citações:** Indica a página exata da informação encontrada.
- **Extração:** Processamento automático via pdfplumber e pandas.



■ Métodos Numéricos, Sympy e Documentação Científica

Equações Diferenciais com Sympy

Você pode usar o Sympy para resolver analiticamente as equações que regem os sistemas elétricos do projeto **Repopulation (UFF)**.

- **Exemplo:** Definir variáveis simbólicas para tensão (V) e corrente (I) e resolver as equações diferenciais de fluxo de carga.
- **Workflow:** Sympy (Solução Simbólica) -> Lambdify -> Numpy (Solução Numérica).

Jupyter Notebooks e Quarto.md

O **Quarto (.qmd)** é a evolução do Jupyter para relatórios técnicos.

- **Por que usar:** Permite misturar Python, equações em LaTeX ($E=mc^2$) e textos explicativos de forma muito superior ao Markdown comum.
- **Aplicação:** Crie um arquivo ``analise_redes.qmd`` dentro do projeto ``Repopulation`` para gerar PDFs acadêmicos automáticos.

■ Roadmap de Projetos Avançados

Para Python

1. **Solver de EDOs Universal:** Um sistema que recebe uma equação diferencial via Sympy e gera um dashboard interativo (Streamlit ou Flet) com a solução numérica e o gráfico de fases.
2. **Agente de IA Matemático:** Integrar o `Pikachu-API` com um agente que utiliza Sympy para verificar cálculos simbólicos enviados pelo usuário.

Para JavaScript

1. **Visualizador de Campos Vetoriais:** Usar **Three.js** para visualizar fluxos de potência em 3D no dashboard do ONS.
2. **Engine de Simulação em Tempo Real:** Criar uma UI em React que simula o comportamento de um PLC (Programmable Logic Controller) usando Web Workers para os cálculos.

■ ■ Próximos Passos Sugeridos

- **Fase 1:** Migrar a documentação do projeto da UFF para **Quarto.md**.
- **Fase 2:** Implementar testes unitários (TDD) no módulo de extração do ``Palkia``.
- **Fase 3:** Refatorar o ``Pikachu-API`` usando Injeção de Dependência (SOLID).



ODS PRIORITÁRIOS